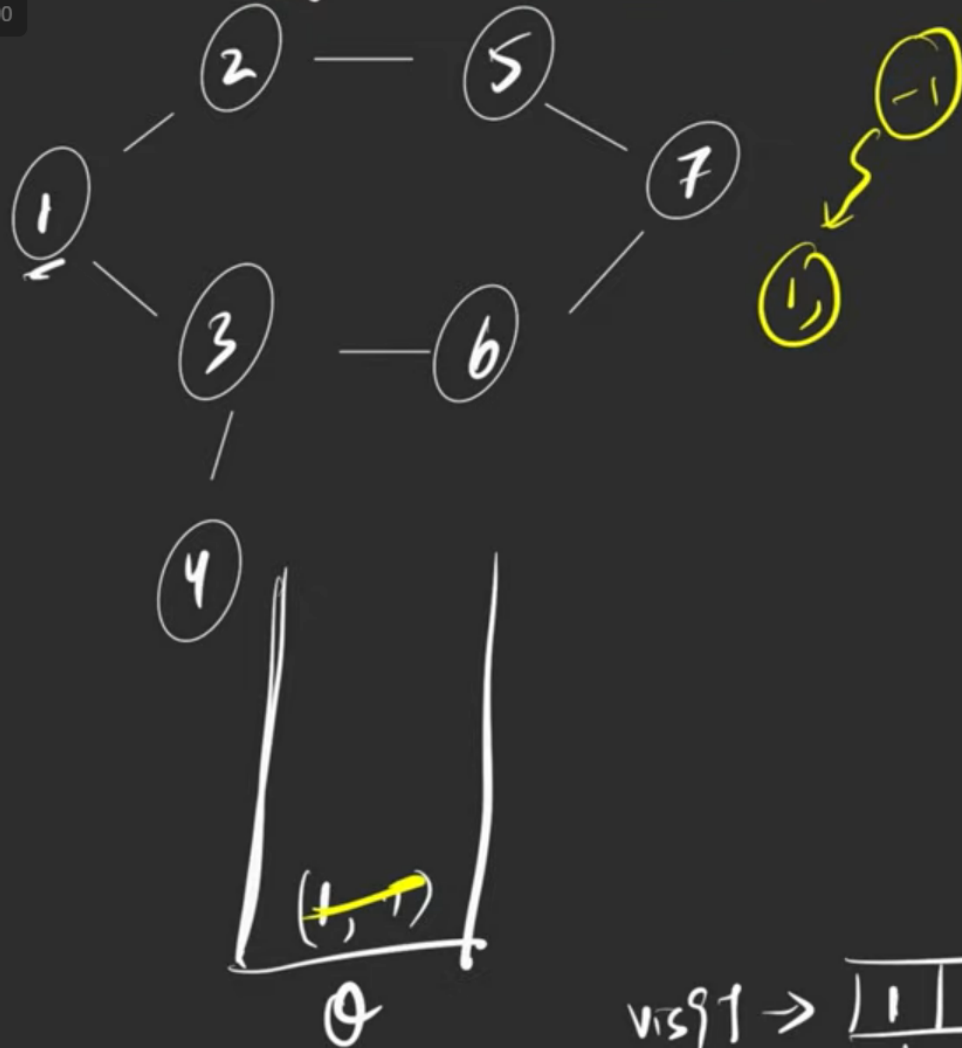


G-11. Detect a Cycle in an Undirected Graph using BFS | C++ | Java

1.90



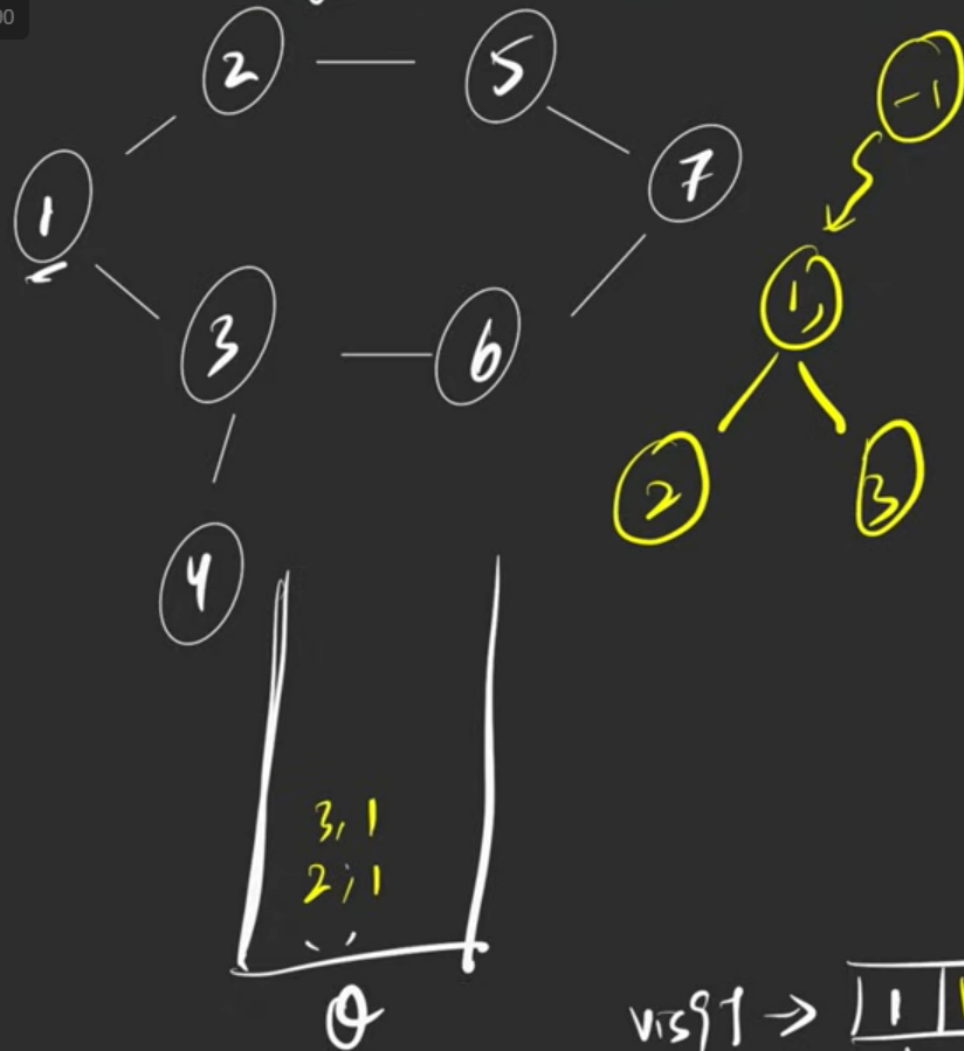
vis[] →

1	0	0	0	0	0	0
1	2	3	4	5	6	7



G-11. Detect a Cycle in an Undirected Graph using BFS | C++ | Java

1.90



adj. list

1 → {2, 3}
2 → {1, 5}
3 → {1, 4, 6}
4 → {3}
5 → {2, 7}
6 → {3, 7}
7 → {5, 6}

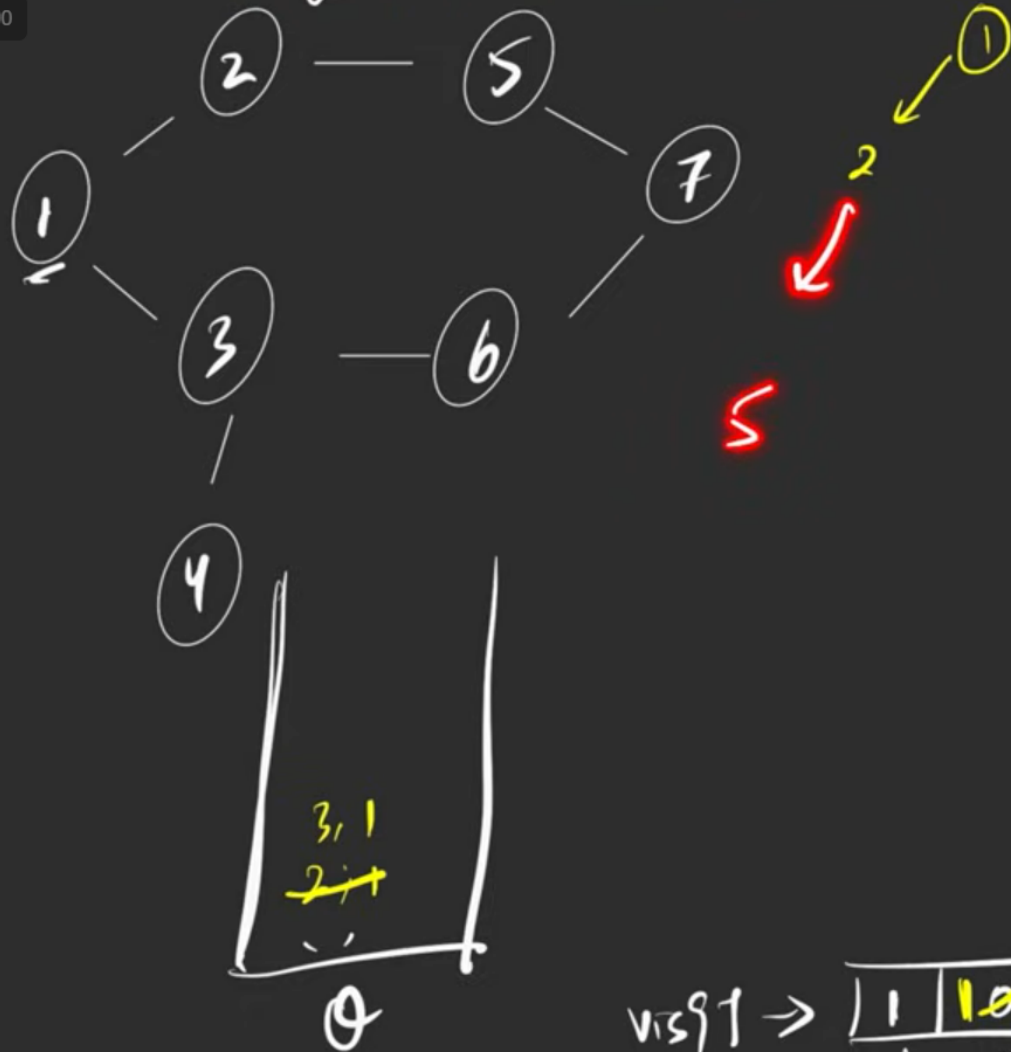
vis[] →

1	1	0	0	0	0	0
1	2	3	4	5	6	7



G-11. Detect a Cycle in an Undirected Graph using BFS | C++ | Java

1.90



adj. list

1	→	{2, 3}
2	→	{1, 5}
3	→	{1, 4, 6}
4	→	{3}
5	→	{2, 7}
6	→	{3, 7}
7	→	{5, 6}

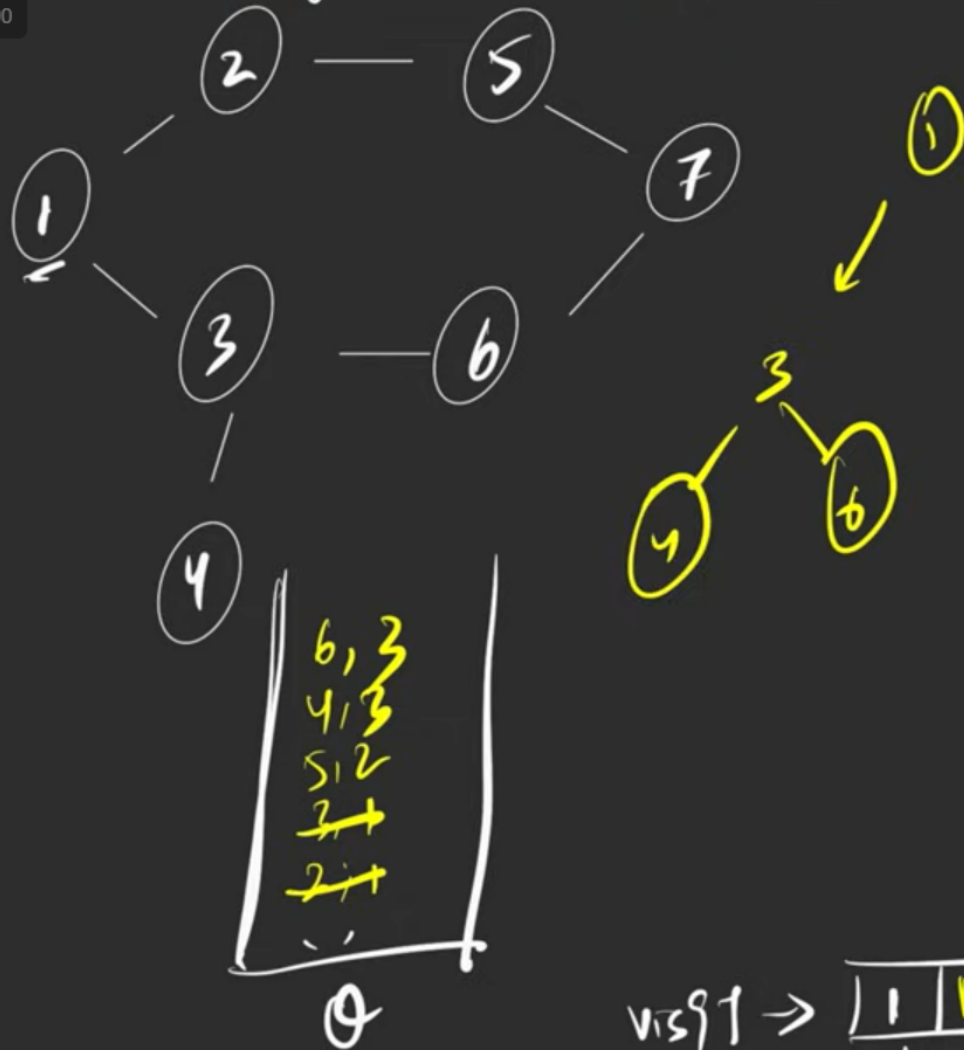
visited →

1	1	1	0	0	1	0	0
	1	2	3	4	5	6	7



G-11. Detect a Cycle in an Undirected Graph using BFS | C++ | Java

1.90



adj. list

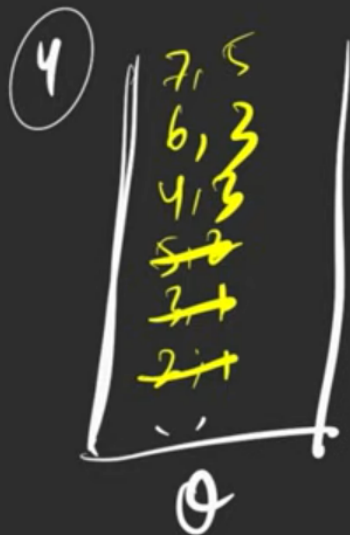
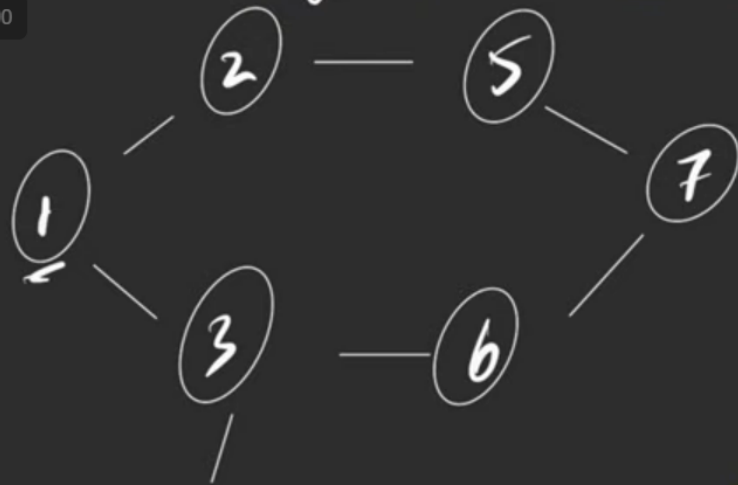
```

1 → { 2, 3 }
2 → { 1, 5 }
3 → { 1, 4, 6 }
4 → { 3 }
5 → { 2, 7 }
6 → { 3, 7 }
7 → { 5, 6 }
    
```

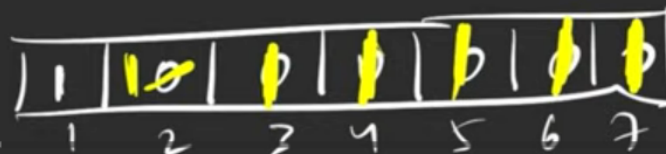


G-11. Detect a Cycle in an Undirected Graph using BFS | C++ | Java

1.90

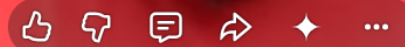


visited →



adj. list

1 → { 2, 3 }
 2 → { 1, 5 }
 3 → { 1, 4, 6 }
 4 → { 3 }
 5 → { 2, 7 }
 6 → { 3, 7 }
 7 → { 5, 6 }



</> Problem

Editorial

Submissions

Comments

C++ (g++ 5.4)



```
1 // } Driver Code Ends
6 class Solution {
7 private:
8 bool detect(int src, vector<int> adj[], int vis[]) {
9     vis[src] = 1;
10    queue<pair<int,int>> q;
11    q.push({src, -1});
12    while(!q.empty()) {
13        int node = q.front().first;
14        int parent = q.front().second;
15        q.pop();
16
17        for(auto adjacentNode: adj[node]) {
18            if(!vis[adjacentNode]) {
19                vis[adjacentNode] = 1;
20                q.push({adjacentNode, node});
21            }
22            else if(parent != adjacentNode) {
23                return true;
24            }
25        }
26    }
27 }
28 public:
29 // Function to detect cycle in an undirected graph.
30 bool isCycle(int V, vector<int> adj[]) {
31     // Code here
32 }
33 };
34
35
```

```
public boolean checkForCycle(int src, int V,
    ArrayList<ArrayList<Integer>> adj, boolean[] vis)
{
    vis[src] = true;
    Queue < Pair > q = new LinkedList < > ();
    q.add(new Pair(src, -1));
    while(!q.isEmpty()) {
        int node = q.peek().first;
        int parent = q.peek().second;
        q.remove();

        for(int adjacentNode: adj.get(node)) {
            if(vis[adjacentNode] == false) {
                vis[adjacentNode] = true;
                q.add(new Pair(adjacentNode, node));
            }
            else if(parent != adjacentNode) {
                return true;
            }
        }
    }
    return false;
}

// Function to detect cycle in an undirected graph.
public boolean isCycle(int V,
    ArrayList<ArrayList<Integer>> adj) {
    boolean vis[] = new boolean[V];
    for(int i = 0; i<V; i++) vis[i] = false;
    for(int i = 0; i<V; i++) {
        if(vis[i] == false) {
            if(checkForCycle(i, V, adj, vis)) return true;
        }
    }
    return false;
}
```

adj = {{}, {2}, {1, 3}, {2}}

Output: 0



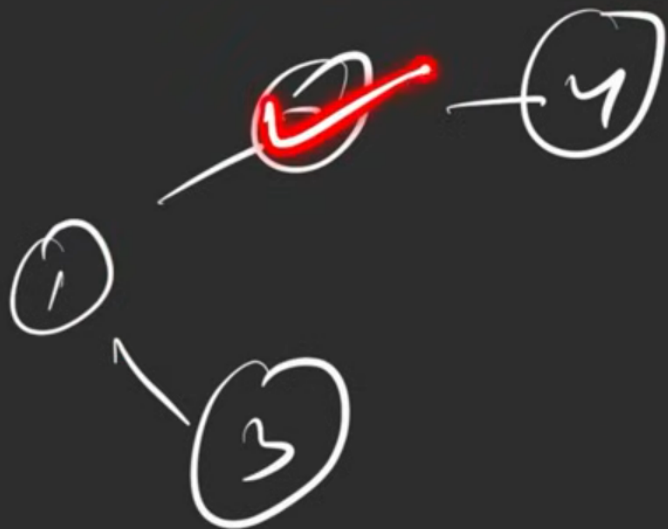
Compile & Run

Submit



G-11. Detect a Cycle in an Undirected Graph using BFS | C++ | Java

1.90



Connected Graph



13:50 / 20:18



1.90

```
for (i = 1; i <= N; i++)  
{  
    if (!vis[i])  
    {  
        if (detectCycle(i) == true)  
            return true;  
    }  
}  
return false
```



G-11. Detect a Cycle in an Undirected Graph using BFS | C++ | Java

Courses Get Hired Events </>POTD

1.90

Practice

321

Problem Editorial Submissions Comments

C++ (g++ 5.4)

```
public boolean checkForCycle(int src, int V,
    ArrayList<ArrayList<Integer>> adj, boolean[] vis)
{
    vis[src] = true;
    Queue < Pair > q = new LinkedList < > ();
    q.add(new Pair(src, -1));
    while(!q.isEmpty()) {
        int node = q.peek().first;
        int parent = q.peek().second;
        q.remove();

        for(int adjacentNode: adj.get(node)) {
            if(vis[adjacentNode] == false) {
                vis[adjacentNode] = true;
                q.add(new Pair(adjacentNode, node));
            }
            else if(parent != adjacentNode) {
                return true;
            }
        }
    }
    return false;
}

// Function to detect cycle in an undirected graph.
public boolean isCycle(int V,
    ArrayList<ArrayList<Integer>> adj) {
    boolean vis[] = new boolean[V];
    for(int i = 0; i < V; i++) vis[i] = false;
    for(int i = 0; i < V; i++) {
        if(vis[i] == false) {
            if(checkForCycle(i, V, adj, vis)) return true;
        }
    }
    return false;
}
```

adj = {{1}, {2}, {1, 3}, {2}}

Output: 0

```
8- bool detect(int src, vector<int> adj[], int vis[]) {
9-     vis[src] = 1;
10-     queue<pair<int,int>> q;
11-     q.push({src, -1});
12-     while(!q.empty()) {
13-         int node = q.front().first;
14-         int parent = q.front().second;
15-         q.pop();

16-         for(auto adjacentNode: adj[node]) {
17-             if(!vis[adjacentNode]) {
18-                 vis[adjacentNode] = 1;
19-                 q.push({adjacentNode, node});
20-             }
21-             else if(parent != adjacentNode) {
22-                 return true;
23-             }
24-         }
25-     }
26-     return false;
27- }
28- }
29- public:
30-     // Function to detect cycle in an undirected graph.
31-     bool isCycle(int V, vector<int> adj[]) {
32-         int vis[V] = {0};
33-         for(int i = 0; i < V; i++) {
34-             if(!vis[i]) {
35-                 if(detect(i, adj, vis)) return true;
36-             }
37-         }
38-         return false;
39-     }
40- };
41- }
```



Compile & Run

Sub



17:01 / 20:18



1.90

}
return false

$$\sum \text{adjacent} = \sum \text{degree} = \textcircled{2E}$$

$$\textcircled{\text{TC}} \rightarrow \textcircled{O(N + 2E)} + O(W)$$

$$\textcircled{\text{SC}} \rightarrow \textcircled{O(N)}, + O(W) \approx O(W)$$

